

Justificativa de Escolha dos Bancos de Dados — Conexão Solidária

FIAP Pós Tech — Tech Challenge Fase 5 Grupo 1: Gustavo Merendi & Thiago Galante
Documento entregável obrigatório conforme requisito do Hackathon.

1. Visão Geral da Estratégia de Dados

A plataforma Conexão Solidária adota o padrão **Database per Service** — cada microserviço possui seu próprio banco de dados isolado. Esta decisão evita acoplamento entre serviços, permite escalonamento independente e alinha-se com os princípios de microserviços.

Foram escolhidos quatro tipos de armazenamento, cada um para um propósito distinto:

Banco	Tipo	Serviços	Propósito Principal
PostgreSQL 16	Relacional (SQL)	Usuários, Campanhas, Doações	Dados transacionais com consistência ACID
Redis 7	In-Memory (NoSQL)	Usuários, Campanhas	Cache de leitura e blacklist de tokens
Elasticsearch	Busca / Documento	Campanhas	Busca textual com tolerância a erro de digitação
AWS DynamoDB	Documento (NoSQL)	Todos	Audit trail e logs estruturados

2. PostgreSQL — Banco Relacional

Por que PostgreSQL?

Consistência ACID é obrigatória para dados financeiros. Uma doação envolve múltiplas operações (inserir doação + atualizar total da campanha) que devem ser atômicas. Se o sistema falhar no meio do processo, não pode haver doações registradas sem atualização do total, nem totais atualizados sem doação registrada.

Relacionamentos são naturais no domínio. Campanhas têm muitas Doações. Usuários realizam muitas Doações. Estas relações são melhor expressas em um modelo relacional com integridade referencial.

Operações atômicas nativas — o Worker de Doações executa `UPDATE campanhas SET valor_arrecadado = valor_arrecadado + @valor` como operação única e atômica, evitando race conditions quando múltiplos eventos de doação chegam em sequência. Antes de aplicar essa atualização, o Worker também confere se o `CorrelationId` do evento já foi processado, o que garante idempotência mesmo em caso de reentrega de mensagem pelo broker — sem essa checagem, uma redelivery do RabbitMQ duplicaria o valor arrecadado.

Excelente suporte no ecossistema .NET via EF Core + Npgsql, com migrations versionadas garantindo evolução controlada do schema.

Por que não outros relacionais?

Alternativa	Motivo da Rejeição
SQL Server	Custo de licença para produção; overhead desnecessário para o porte do MVP
MySQL	Tipo uuid nativo do Postgres é mais ergonômico que o equivalente em MySQL (que exige CHAR(36) ou BINARY(16)); Postgres também implementa mais do padrão SQL (CTEs recursivas, window functions maduras desde antes da versão 8) e oferece JSONB indexável via GIN, útil para os payloads de auditoria
SQLite	Sem suporte adequado a múltiplas conexões concorrentes (API + Worker escrevendo ao mesmo tempo)

Modelo de Dados por Serviço (simplificado)

Usuários (Usuario): nome completo, e-mail (value object, único), CPF (value object, criptografado com AES-256), hash de senha (BCrypt), perfil (DOADOR ou GESTOR_ONG).

Campanhas (Campanha): título, descrição, data de início, data de fim, meta financeira, valor arrecadado, status (ATIVA, CONCLUIDA, CANCELADA).

Doações (Doacao, persistida pelo Worker): dados do doador e da campanha replicados no momento do evento (nome, e-mail, CPF, título da campanha), valor da doação e CorrelationId — usado tanto para rastreamento ponta a ponta quanto como chave de idempotência do processamento.

3. Redis — Cache e Controle de Sessão

Por que Redis?

Velocidade sub-milissegundo — dados em memória acessados em microssegundos, ideal para operações de alta frequência como validar se um token foi revogado a cada requisição autenticada.

TTL nativo — permite que chaves expirem sozinhas, sem necessidade de job de limpeza.

Não é usado como banco principal — Redis é volátil por padrão. É usado apenas para dados regeneráveis (cache reconstruído a partir do PostgreSQL) ou de vida curta (blacklist de token, que só precisa durar até o token expirar naturalmente).

Casos de Uso Reais no Projeto

Chave	TTL	Valor	Serviço
blacklist: {token}	tempo restante até a expiração do JWT	"revogado"	Usuários — logout invalida o token antes da expiração natural
usuario:{email}	30 min	dados do usuário autenticado	Usuários — evita releitura do Postgres a cada requisição
campanha:{guid}	configurável (Cache:CampanhaA tivaTTLSeconds)	dados da campanha	Campanhas — invalidado explicitamente a cada alteração/conclusão/ cancelamento
campanhas:ativas :pagina: {n}:tamanho: {tamanho}	configurável	página do painel de transparência	Campanhas — listagem pública

Nota: a autenticação do projeto não utiliza refresh tokens — apenas o token de acesso (JWT) e a blacklist de revogação acima. A idempotência do processamento de doações é resolvida checando o CorrelationId diretamente no PostgreSQL (ver Seção 2), e não por uma chave dedicada no Redis.

Teorema CAP

Redis, na configuração deste projeto (instância única, sem cluster), prioriza disponibilidade e desempenho sobre consistência forte. Para os casos de uso acima, eventual consistency é aceitável — o pior cenário é um cache-miss (que recarrega do PostgreSQL) ou, no caso da blacklist, uma janela de milissegundos entre a revogação e sua efetivação.

4. AWS DynamoDB — Audit Trail e Logs Estruturados

Por que DynamoDB?

Schema-less para dados heterogêneos — cada evento de auditoria ou linha de log tem estrutura diferente. Com DynamoDB não há migrations: cada item pode carregar atributos distintos.

Escala automática sem configuração — modo PAY_PER_REQUEST, sem necessidade de provisionar capacidade ou fazer sharding manual à medida que o volume de logs cresce.

TTL nativo para retenção — os itens carregam um atributo TTL, permitindo expiração automática sem job de limpeza.

Sem relacionamentos — logs e eventos de auditoria não têm JOINS. Usar PostgreSQL para isso desperdiçaria a capacidade relacional do banco principal em algo que não precisa dela.

Modelo de Dados Real (duas tabelas)

O projeto usa **duas tabelas DynamoDB distintas**, cada uma otimizada para um padrão de acesso diferente:

audit-log — trilha de auditoria de alterações em entidades (gravada automaticamente por um interceptor do EF Core sempre que uma entidade é criada/alterada/excluída):

Chave primária: PK (hash) + SK (range)

Índice secundário global: ResourceIdIndex (hash: ResourceId, range: SK)

TTL: atributo "TTL"

O GSI ResourceIdIndex permite consultar todo o histórico de alterações de uma entidade específica (ex: um usuário ou uma campanha) sem depender de um scan completo da tabela.

app-log — logs estruturados de aplicação e eventos de domínio, escritos quando `SAVE_LOGS=true`:

Chave primária: CorrelationId (hash) + Timestamp (range)

Índice secundário global: GSI_EventSourcing (hash: Type, range: Timestamp)

TTL: atributo "TTL"

O atributo Type distingue log comum (0) de evento de domínio (1), permitindo consultas por tipo ordenadas por tempo — o padrão que sustenta a visão de “event sourcing” usada no dashboard do Grafana (via o serviço DynamoPgProxy, que traduz essas consultas para o formato que o Grafana entende).

Por que não MongoDB?

Critério	DynamoDB	MongoDB
Operacional	Serverless (zero configuração)	Requer gerenciamento de cluster
Escala	Automática (PAY_PER_REQUEST)	Manual ou via Atlas gerenciado
TTL nativo	Sim	Sim
Integração AWS	Nativa (IAM, sem driver externo)	Via driver, sem integração nativa
Custo no projeto	Free tier generoso	Exige cluster próprio

Como o projeto já roda inteiramente na AWS (com DynamoDB Local para desenvolvimento), DynamoDB é a escolha operacionalmente mais simples e coerente com o restante da infraestrutura.

5. Elasticsearch — Busca com Tolerância a Erros

Por que Elasticsearch e não PostgreSQL LIKE/ILIKE?

Critério	Elasticsearch	PostgreSQL LIKE/ILIKE
Tolerância a erro de digitação	Sim (Fuzziness: AUTO, distância de edição)	Não
Performance em texto livre	Índice invertido	Scan sequencial
Ranking por relevância	Sim (BM25, nativo do motor)	Não

O campo `Titulo` e `Descricao` das campanhas são indexados como texto pesquisável, e a busca usa `MultiMatch` com `Fuzziness ("AUTO")` — ou seja, tolera pequenos erros de digitação (distância de edição), permitindo que “campahna” ainda encontre “campanha”. O campo de status usa um *normalizer* de lowercase para permitir filtro case-insensitive.

Importante: a implementação atual usa o *analyzer* padrão do Elasticsearch com correspondência aproximada por distância de edição — não há um analisador morfológico da língua portuguesa (`portuguese`) configurado no índice. Isso significa que a tolerância é a erros de digitação (ex: “campahna” — “campanha”), mas não a variações morfológicas mais distantes na forma escrita (ex: relacionar “criança” a “criançada” não seria coberto apenas pela distância de edição). Configurar um `analyzer portuguese` no mapeamento do índice é uma melhoria futura natural, caso se queira busca morfológica completa.

PostgreSQL não foi projetado para esse tipo de busca — é excelente para o que faz, mas ranking e tolerância a erro em texto livre são resolvidos de forma muito mais direta pelo Elasticsearch.

6. Comparativo — Teorema CAP por Armazenamento

Armazenamento	Tendência CAP	Justificativa
PostgreSQL	Consistência forte	Dados financeiros — consistência é obrigatória
Redis	Disponibilidade	Cache e blacklist — eventual consistency é aceitável
DynamoDB	Disponibilidade, com opção de leitura fortemente consistente por partição	Audit trail e logs — disponibilidade preferível, sem necessidade de transação entre itens
Elasticsearch	Disponibilidade	Busca — índice pode estar alguns segundos atrás da escrita no Postgres, o que é aceitável para essa funcionalidade

7. Decisão Final

A escolha de múltiplos armazenamentos diferentes pode parecer complexidade adicional, mas cada um resolve um problema específico de forma mais adequada do que uma solução única resolveria:

- **PostgreSQL** quando precisamos de **garantias ACID e relacionamentos** (dados financeiros e transacionais)
- **Redis** quando precisamos de **velocidade e expiração automática** (cache de leitura e blacklist de tokens)
- **Elasticsearch** quando precisamos de **busca tolerante a erros em texto livre** (painel público de campanhas)
- **DynamoDB** quando precisamos de **escala automática, schema flexível e retenção via TTL** (auditoria e logs)

Usar PostgreSQL para tudo seria operacionalmente mais simples, mas resultaria em: degradação de performance no cache (banco relacional não é otimizado para isso), necessidade de migrations para cada novo formato de evento de auditoria, e busca de texto livre sem tolerância a erro no painel de campanhas. A escolha poliglota é justificada pelos ganhos concretos em cada frente, e está alinhada com o princípio de microsserviços de que cada serviço escolhe o armazenamento mais adequado ao seu próprio domínio.